UNIT II: Arrays, Searching, and Sorting

By – Dr. Dipalee D. Rane

(COMPUTER ENGG, DYPCOE, AKURDI, PUNE)

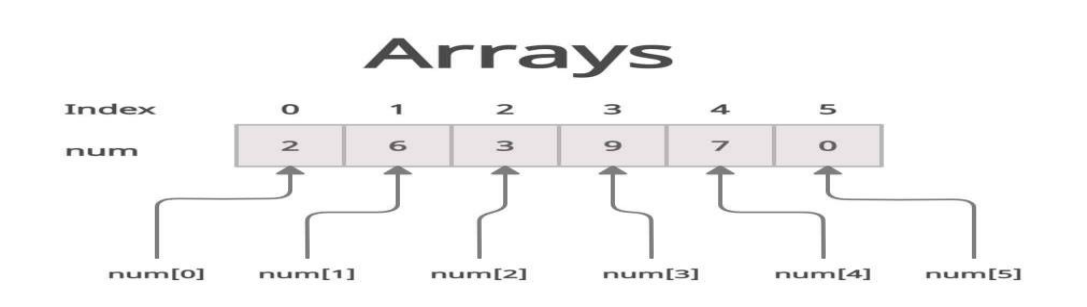
Overview of Array

What is an Array?

An **array** is a linear data structure that stores elements **of the same data type** in a **contiguous block of memory**, accessed using indices.

Key Characteristics

- Fixed size (static allocation)
- Elements are indexed from **0 to n-1**
- Random access enabled (O(1) time complexity)
- Efficient storage and access for homogeneous data



Overview of Array

Types of Arrays

- **One-Dimensional Array (1D)**: Simple list of elements
  (e.g., int arr[5] = {1, 2, 3, 4, 5})
- **Two-Dimensional Array (2D)**: Matrix-like structure
  (e.g., int matrix[3][3])
- **Multidimensional Array (nD)**: Higher-level data representations
  (e.g., tensor-like structures in ML)

Real-World Applications

- Storing marks of students
- Representing matrices and images
- Lookup tables
- Buffering and caching systems

Array as an Abstract Data Type

What is an Abstract Data Type (ADT)?

- An **ADT** is a **theoretical model** that defines:
- The **data** to be stored
- The **operations** allowed on that data **without** specifying the internal implementation

Array as an ADT

- An array as an ADT defines:
- A collection of elements of the same type
- Fixed size (defined at creation)
- Indexed access using position

Array as an Abstract Data Type

Advantages of Array as ADT

- Random Access
- Simplicity
- Memory Locality
- Efficient Iteration

Disadvantages of Array as ADT

- Fixed Size
- Costly Insertions/Deletions
- Wasted Memory
- Inefficient for Dynamic Operations

Operations on Arrays

1. Traversal

- Visiting each element of the array from start to end
- Used for displaying, summing, or processing all elements
- 🕒 Time Complexity: O(n)

2. Insertion

- Adding an element at a specific index
- Requires shifting elements to the right
- 🕒 Time Complexity:

• Best Case (end insertion): O(1)

• Worst Case (front insertion): O(n)

Operations on Arrays

— 3. Deletion

- Removing an element from a specific index
- Requires shifting elements to the left to fill the gap
- 🕒 Time Complexity: O(n)

📝 4. Updating

- Replacing an element at a given index with a new value
- 🕒 Time Complexity: O(1)

🔍 5. Searching

- Finding the position of an element
- Types: Linear Search O(n), Binary Search O(log n) (if sorted)

Storage Representation

How Arrays are Stored in Memory

- Arrays are stored in **contiguous blocks of memory**
- Each element is placed at a memory address calculated using:

$$\text{Address} = \text{Base Address} + (i \times \text{Size of data type})$$

1D Array Storage

- Elements are laid out sequentially
- Enables **constant-time access** using index

Name	roll[0]	roll[1]	roll[2]	roll[3]	roll[4]	roll[5]	roll[6]	roll[7]
Values	12	45	32	23	17	49	5	11
Address	1000	1002	1004	1006	1008	1010	1012	1014

1-D Array memory arrangement

Multidimensional Arrays [2D, nD]

What is a Multidimensional Array?

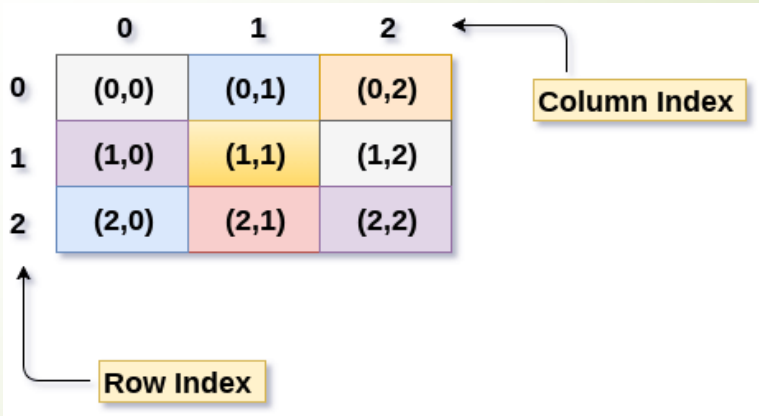
- A **multidimensional array** is an array of arrays. It allows storing data in **table-like (2D)** or **cube-like (nD)** structures.

2D Array

- Represents a **matrix** with rows and columns
- Declaration example in JAVA:

```
int arr[][] = new int[2][3];
```

Accessed using: matrix[i][j]



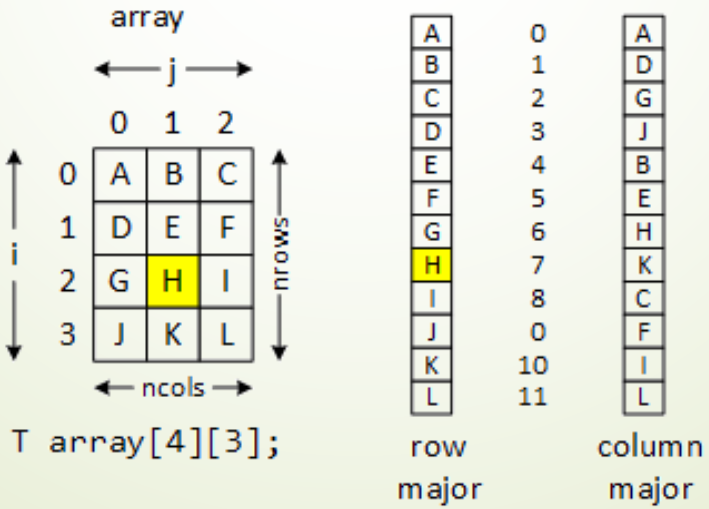
Applications:
Image processing, Graph representations, Matrix operations

Multidimensional Arrays [2D, nD]

2D Array Storage

- There are two common representations:

1. **Row-Major Order**
   All elements of the first row are stored first, followed by second, and so on.
2. **Column-Major Order**
   All elements of the first column are stored first, followed by second, and so on.

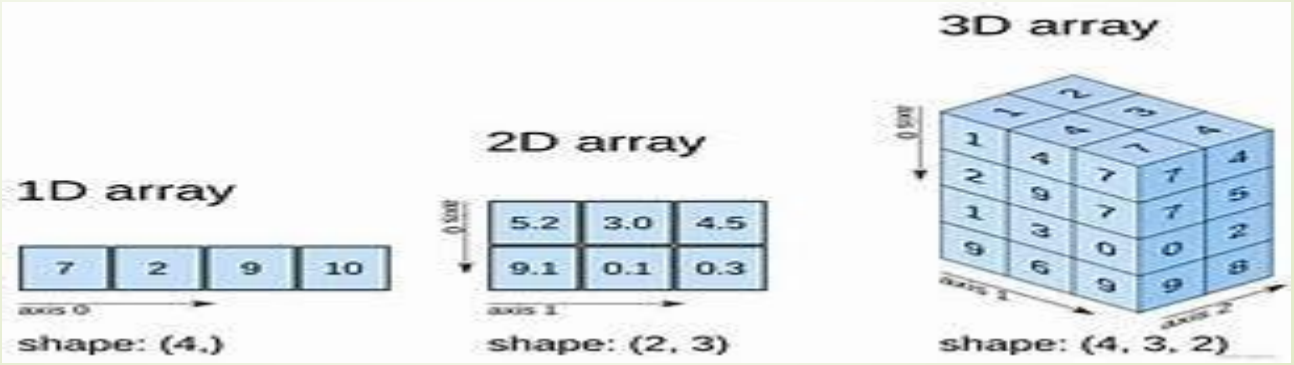


Multidimensional Arrays [2D, nD]

nD Array

- Extension of 2D array into multiple dimensions
- Useful in advanced simulations, tensor computations, etc.
- **Example**: int tensor[3][4][2];

Applications:
Machine learning (tensors), physics simulations, 3D graphics



Sparse Matrix Representation using 2D Arrays

What is a Sparse Matrix?

- A **sparse matrix** is a matrix in which **most elements are zero**. Storing all elements wastes memory and computation.

Why Use Sparse Representation?

- Efficient **storage** of large matrices with few non-zero values
- Reduces **space complexity** significantly
- Speeds up **matrix operations** by ignoring zeros

Common Use Cases

- Graph algorithms (adjacency matrix)
- Machine learning (feature vectors)
- Image compression

Sparse Matrix Representation using 2D Arrays

Sparse Matrix using 2D Array

- One popular method is the **Triplet Representation** (Row, Column, Value):

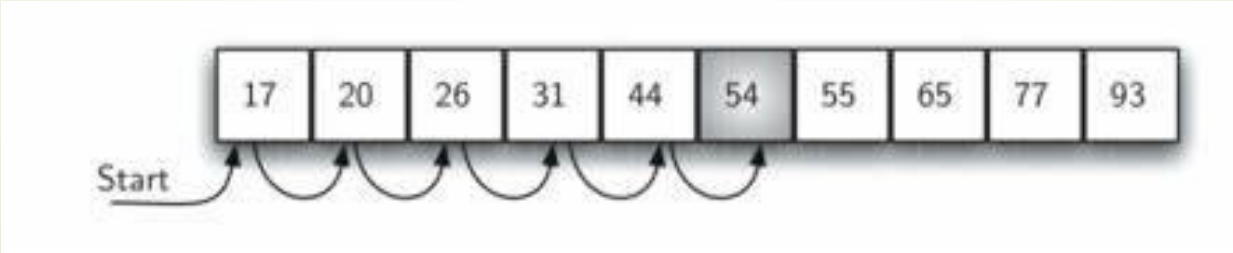
Row	Col	Value
0	2	5
1	0	8
3	1	6

- Here, only non-zero elements are stored along with their positions.

Searching Techniques

1. Sequential Search (Linear Search)

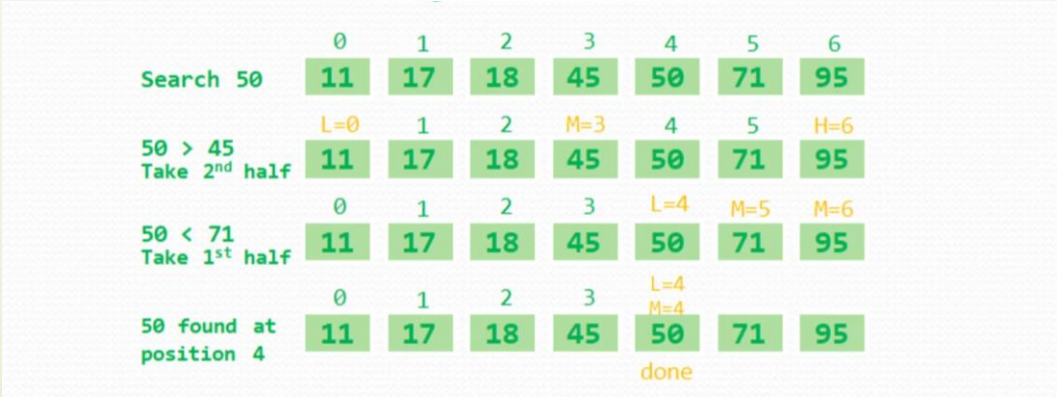
- Check each element one by one
- Works on **unsorted arrays**
- Time Complexity: O(n)
- Best Case: First element
- Worst Case: Last element or not found



Searching Techniques

2. Binary Search

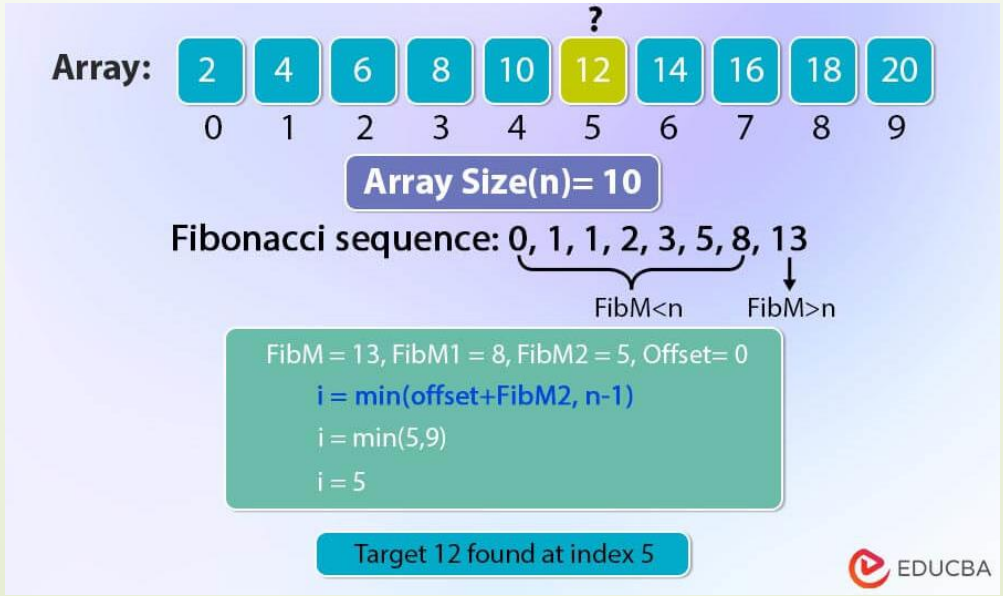
- Works only on **sorted arrays**
- Repeatedly divides the search space in half
- Time Complexity: O(log n)
- Requires sorting beforehand



Searching Techniques

3. Fibonacci Search

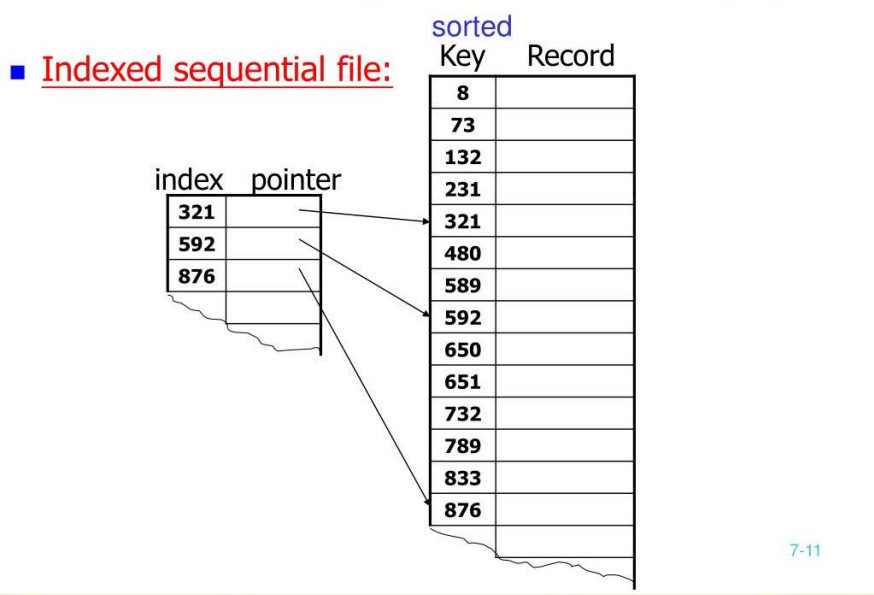
- Based on Fibonacci numbers for split positions
- Suitable for large sorted arrays
- Time Complexity: O(log n)
- Uses less division than binary search (faster on some machines)



Searching Techniques

4. Indexed Sequential Search

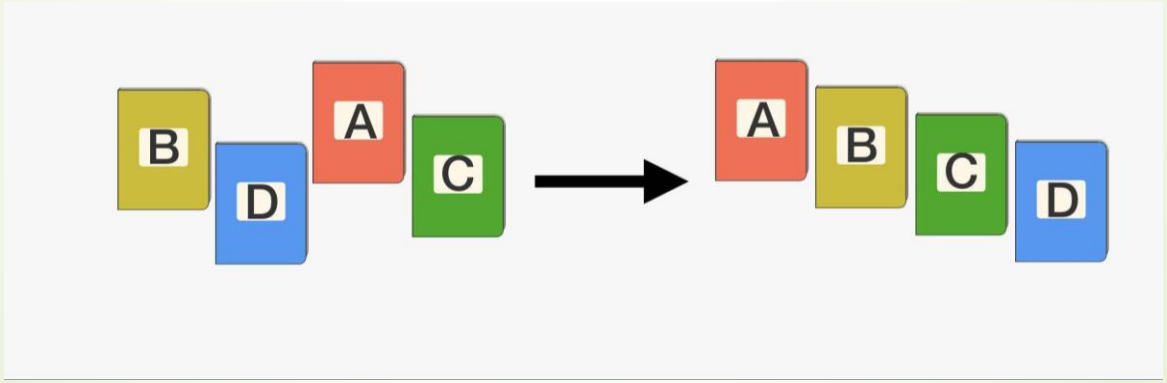
- Combine **indexing** with **linear search**
- Use a separate **index table** for faster access
- Efficient when working with large sorted datasets



Sorting Concepts

Why Sorting?

- Sorting organizes data in a specific order (ascending/descending), making searching and processing **faster and more efficient**.



Sorting Concepts

Key Concepts in Sorting

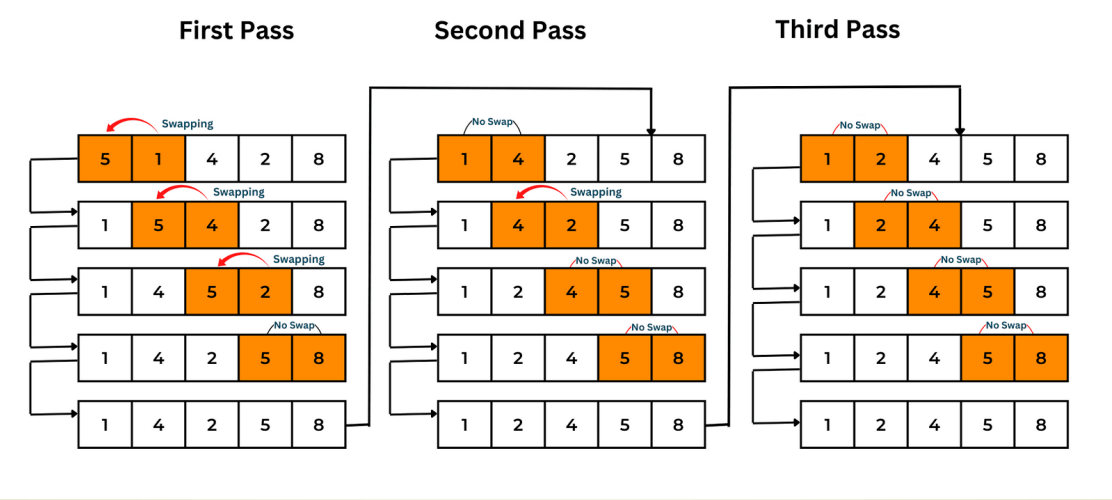
- **1. Stability**
- A sorting algorithm is **stable** if it preserves the **relative order of equal elements**.
- Example: In student list, two students with the same marks maintain their input order

- **2. Efficiency**
- Based on **time** and **space complexity**
- Depends on best, average, and worst-case analysis

Sorting Algorithms

1. Bubble Sort

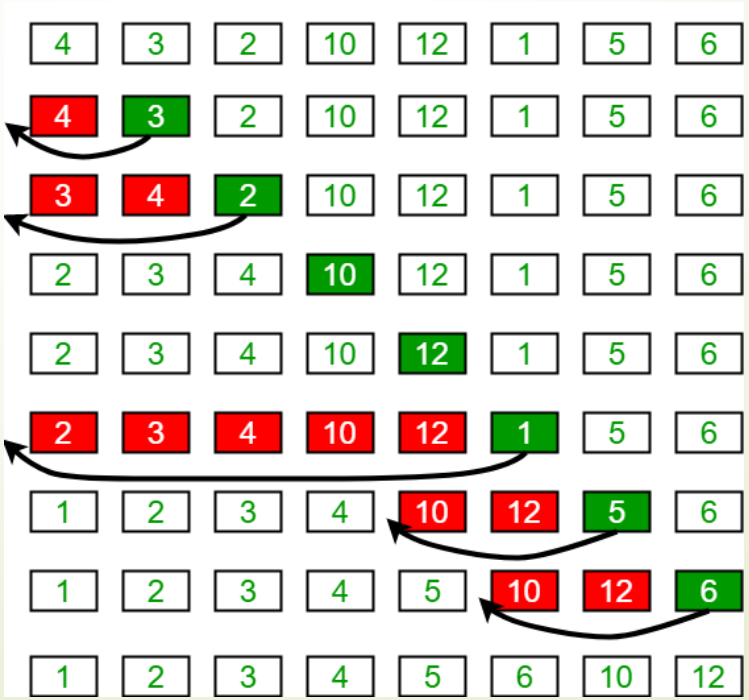
- Compares adjacent elements and swaps if needed
- Repeats until the list is sorted
  Time: O(n²)
  Stable
  Easy but inefficient for large datasets



Sorting Algorithms

2. Insertion Sort

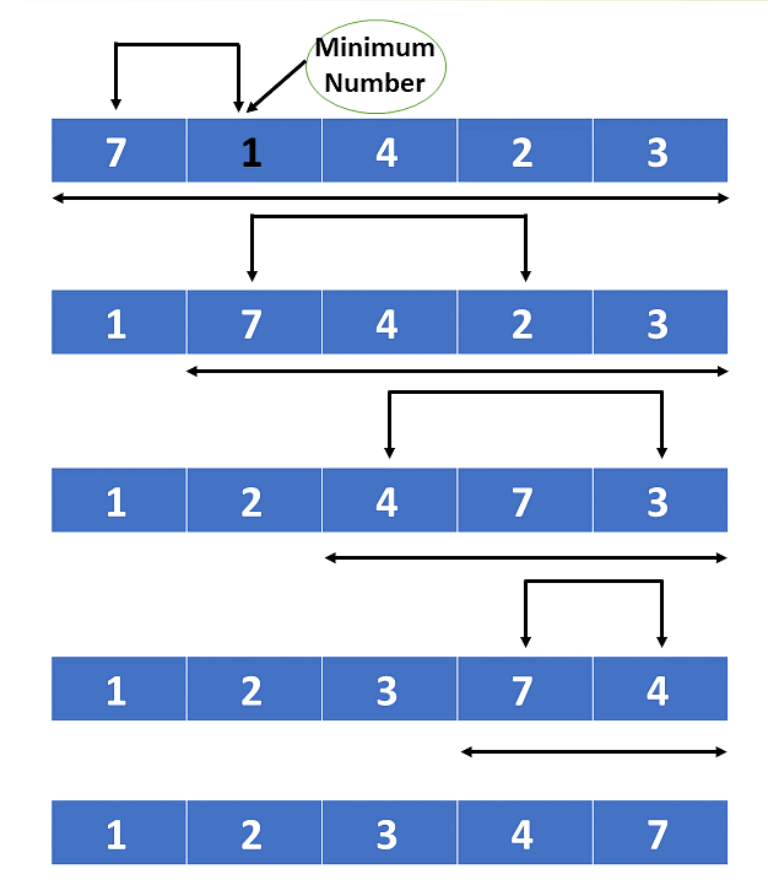
- Builds sorted list by inserting one element at a time
  Time: $O(n^2)$
  Stable
  Efficient for small or nearly sorted arrays



Sorting Algorithms

3. Selection Sort

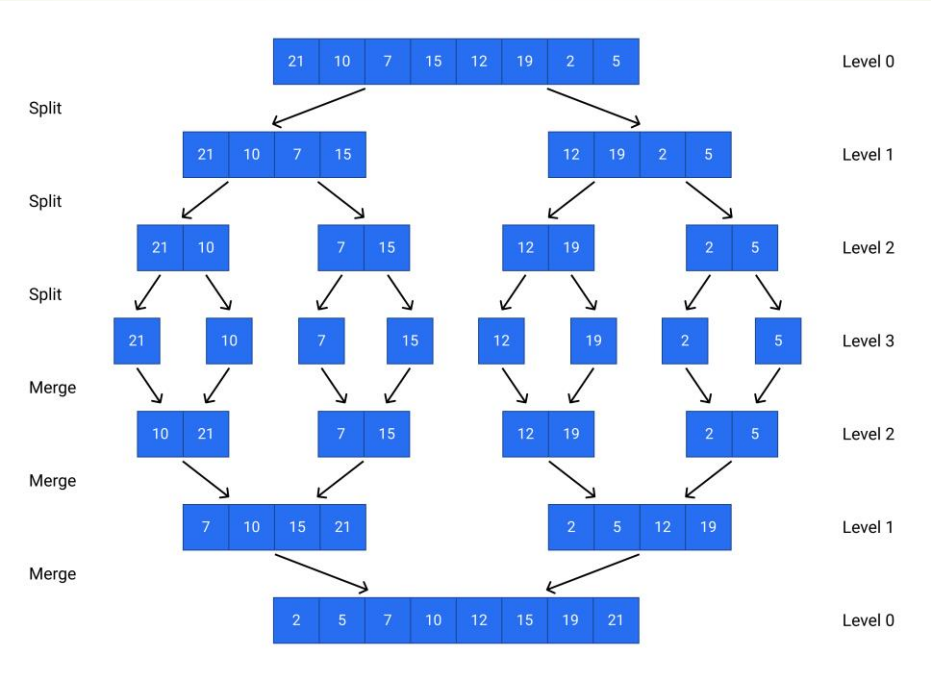
- Repeatedly selects the minimum element and places it at the front
  Time: $O(n^2)$
  Not Stable
  Simple, but not efficient



Efficient Sorting Algorithms

1. Quick Sort

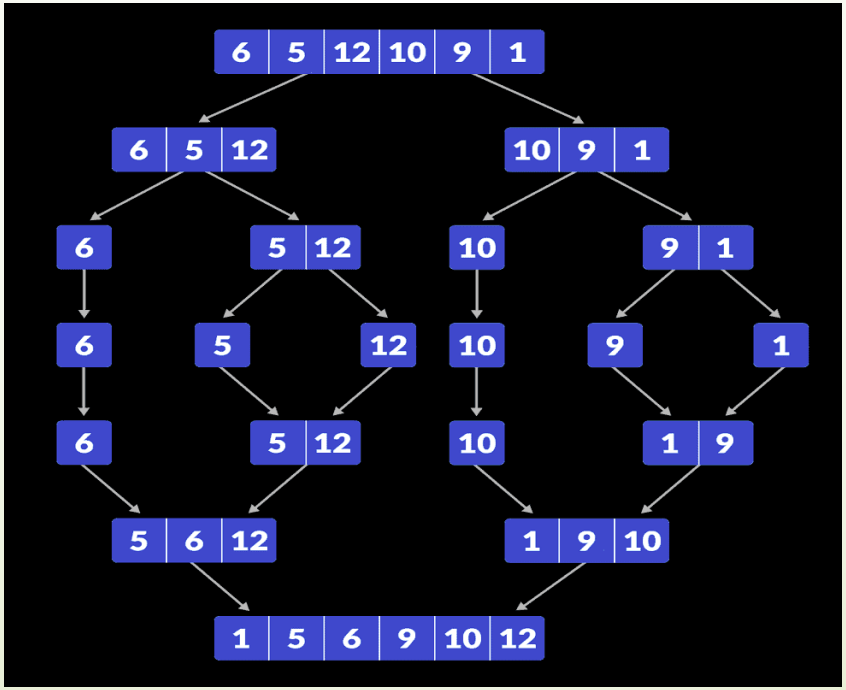
- Divide-and-conquer using pivot-based partitioning
  Time: O(n log n) average, O(n²) worst
  Not Stable
  Very efficient and widely used



Efficient Sorting Algorithms

2. Merge Sort

- Divide the array, sort halves, and merge
  Time: O(n log n)
  Stable
  Uses extra space, but reliable and consistent



Exemplar / Case Study: Social Network Adjacency Matrix

- Use Case: Representing friendship connections in social media platforms.
- Data Structure Used: 2D Adjacency Matrix.
- Matrix Element: a[i][j] = 1 if user i is friends with user j, else 0.
- For millions of users, the matrix is very sparse.
- Sparse matrix techniques help save memory and improve performance.
- Useful in applications like friend suggestion, community detection, etc.

Text Books

1. Fundamentals of Data Structures in C – by Ellis Horowitz, Sartaj Sahni, Susan Anderson-Freed Universities Press

2. Data Structures Through C - by Yashavant Kanetkar BPB Publications

NPTEL Links

1. Sorting and Searching by IIT Kharagpur

   https://www.youtube.com/watch?v=ZwFkaGTt4Xw

2. NPTEL Course - NOC: Programming, Data Structures and Algorithms using Python

   https://nptel.ac.in/courses/106106145